

次世代 3D 數位著裝與神經布料物理系統 (PoC 開發計畫)

針對 CLO3D 傳統物理引擎瓶頸之 AI/WebGPU 破局解方與 GitHub Codespace Agent 執行指南

專案目標：震撼級 Web 端 3D 著裝 PoC (Proof of Concept)

核心技術：Python / PyTorch / Three.js / WebGPU / ONNX

核心特色：多層套娃 Avatar (SDF) + 應力場微褶皺 + 風場波浪搖曳

執行環境：GitHub Codespace Agent Automated Build

一、PoC 核心展示目標與技術破局點

本 PoC 的終極目標是建立一個基於 Web 端 (Three.js + WebGPU) 的高視覺衝擊力 Demo，直接擊中傳統 3D 服飾軟體 (如 CLO3D) 在多層穿搭穿模、缺乏重量感與受力微褶皺的技術瓶頸。

✨ 震撼對方的三大對比 Feature (Kill Shots)

- 1. 應力張力場與動態微褶皺 (Dynamic Stress-Driven Micro-Wrinkles)**：在肩膀、胸背等受力支撐點，根據重力與骨架動態即時生成真實的 Normal/Displacement 應力張力圖，展現真實布料緊繃與垂墜弧線。
- 2. 多層級套娃 Avatar 距離場 (Multi-Layer Signed Distance Field / Layered SDF)**：解決內衣、襯衫、厚外套三層疊加時的穿模問題，外層布料能根據內層厚度自然「被撐起」，無硬碰撞拉扯感。
- 3. 輕量化神經布料風場與絹光 PBR 渲染 (Neural Wind & Sheen PBR)**：利用自訂 Shader 結合空氣動力學波浪函數，讓靜止與運動狀態下的衣擺呈現輕盈自然擺動。

二、系統整體架構圖 (System Architecture)

```
+-----+ | [ Python /  
PyTorch Offline Pipeline ] | | 1. Layered Avatar SDF Generator --> 2. Neural Tension & Stress Field  
Model | | (SMPL-X Avatar + Clothes Mesh) (Predict Tension Map / Normal Displacement) | | | | ▼  
Export to ONNX / JSON |  
+-----+ |  
+-----v-----+ | [ Three.js +  
WebGPU Real-time Frontend ] | | — Avatar Engine: SMPL-X Skeleton Rigging & Dynamic Layered SDF  
Offset | | — Physics Engine: Real-time Vertex Stress Deformation & Micro-Wrinkle Shader | | —  
Material Engine: PBR Sheen Shader + Custom FFT Dynamic Wind Wave Function |  
+-----+
```

三、GitHub Codespace Agent 逐步自動化開發指令腳本

Agent 可直接執行以下目錄結構設定與代碼生成。全專案設計為單一可獨立執行的 Web 展示專案，後端 Python 提供模型導出腳本，前端使用 Vite + Three.js 進行高效渲染。

Step 1: 建立專案目錄結構與環境 setup

```
# 建立專案目錄
mkdir -p cloth-sim-poc/{src,public/assets,python_pipeline}
cd cloth-sim-poc

# 初始化 Node.js 專案並安裝 Three.js 與開發工具
npm init -y
npm install three three-stdlib onnxruntime-web dat.gui
npm install -D vite glslify

# Python 管道環境 setup
cd python_pipeline
python3 -m venv venv
source venv/bin/activate
pip install torch numpy trimesh scipy onnx
cd ..
```

Step 2: 核心 Shader 寫入 (張力應力場與微褶皺法線貼圖)

建立 `src/shaders/ClothMaterialShader.js`，用於在 `Three.js` 中即時根據受力權重計算微褶皺與 PBR 光影：

```

import * as THREE from 'three';

export const ClothCustomShader = {
  uniforms: {
    uTime: { value: 0 },
    uGravityTension: { value: 1.0 },
    uWindSpeed: { value: 0.5 },
    uBaseColor: { value: new THREE.Color(0x2b3a4a) },
    uSheenColor: { value: new THREE.Color(0x88c0d0) }
  },
  vertexShader: `
    uniform float uTime;
    uniform float uGravityTension;
    uniform float uWindSpeed;

    attribute float aStressWeight; // 受力點權重 (肩膀/胸圍等)
    varying vec3 vNormal;
    varying vec2 vUv;
    varying float vStress;

    void main() {
      vUv = uv;
      vNormal = normalMatrix * normal;
      vStress = aStressWeight * uGravityTension;

      vec3 pos = position;

      // 動態風場微幅波浪 (空氣阻力擺動)
      float wave = sin(pos.y * 5.0 + uTime * 3.0 * uWindSpeed) * 0.02 * (1.0 - aStressWeight);
      pos.x += wave;
      pos.z += cos(pos.x * 4.0 + uTime * 2.0) * 0.015 * (1.0 - aStressWeight);

      // 受力點的垂直拉伸與垂墜弧線
      pos.y -= vStress * 0.03;

      gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
    }
  `,
  fragmentShader: `
    uniform vec3 uBaseColor;
    uniform vec3 uSheenColor;
    varying vec3 vNormal;
    varying vec2 vUv;
    varying float vStress;

    void main() {
      vec3 N = normalize(vNormal);
      vec3 L = normalize(vec3(1.0, 2.0, 1.5));
      float diff = max(dot(N, L), 0.2);

      // 生成微褶皺高頻法線細節 (受力張力越大，褶皺密度越高)
      float wrinkle = sin(vUv.y * 120.0 + vStress * 20.0) * cos(vUv.x * 80.0) * vStress;
      vec3 finalColor = uBaseColor * (diff + wrinkle * 0.2);

      // 絹光 PBR (Sheen Effect) 增加輕盈與面料高級感
      float viewAngle = max(dot(N, vec3(0.0, 0.0, 1.0)), 0.0);
      float sheen = pow(1.0 - viewAngle, 3.0);
      finalColor += uSheenColor * sheen * 0.6;
    }
  `
};

```

```
gl_FragColor = vec4(finalColor, 1.0);  
}  
};
```

四、多層套娃 (Layered SDF) 幾何防穿模運算 logic

傳統 3D 服飾穿模的主因是層級間無距離感知。在此邏輯中，內層 Avatar / 衣物會輸出動態 Signed Distance Field (SDF) 陣列，外層衣物頂點在渲染前執行動態推離補償：

$$P_{outer}' = P_{outer} + \hat{n} \cdot \max(0, d_{min} - SDF(P_{outer}))$$

其中 d_{min} 為安全層級間距（如內衣 1.0mm、襯衫 2.5mm、外套 5.0mm）， $\hat{n}$ 為表面法線向量。該邏輯可在 Web Workers 中每幀進行 60 FPS 的輕量幾何修正。

五、PoC Web 核心開發腳本 (Three.js 應用程式主入口)

建立 src/main.js，Agent 直接組裝出帶有控制面板 (GUI)、分割對比畫面與動態風場/重力拉桿的即時展示場景：

```

import * as THREE from 'three';
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js';
import * as dat from 'dat.gui';
import { ClothCustomShader } from './shaders/ClothMaterialShader.js';

// 1. 初始化 Scene, Camera, Renderer
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x0f172a);

const camera = new THREE.PerspectiveCamera(45, window.innerWidth / window.innerHeight, 0.1, 100);
camera.position.set(0, 1.2, 3.0);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
document.body.appendChild(renderer.domElement);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;

// 2. 建立多層套娃模型 (Avatar Body + Outer Dress)
const avatarGeo = new THREE.CylinderGeometry(0.25, 0.18, 1.4, 32);
const avatarMat = new THREE.MeshStandardMaterial({ color: 0x334155, roughness: 0.8 });
const avatar = new THREE.Mesh(avatarGeo, avatarMat);
avatar.position.y = 0.7;
scene.add(avatar);

// 建立衣物 Mesh 並寫入受力點權重 (aStressWeight)
const dressGeo = new THREE.CylinderGeometry(0.27, 0.55, 1.2, 64, 64, true);
const posAttr = dressGeo.attributes.position;
const stressWeights = new Float32Array(posAttr.count);

for (let i = 0; i < posAttr.count; i++) {
  let y = posAttr.getY(i);
  // 頂部 (肩膀受力點) 權重最高, 底部垂墜擺動
  stressWeights[i] = Math.pow((y + 0.6) / 1.2, 2.0);
}
dressGeo.setAttribute('aStressWeight', new THREE.BufferAttribute(stressWeights, 1));

const dressMat = new THREE.ShaderMaterial({
  vertexShader: ClothCustomShader.vertexShader,
  fragmentShader: ClothCustomShader.fragmentShader,
  uniforms: THREE.UniformsUtils.clone(ClothCustomShader.uniforms),
  side: THREE.DoubleSide
});

const dress = new THREE.Mesh(dressGeo, dressMat);
dress.position.y = 0.7;
scene.add(dress);

// 光照設定
const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
scene.add(ambientLight);
const dirLight = new THREE.DirectionalLight(0x38bdf8, 1.2);
dirLight.position.set(2, 4, 3);
scene.add(dirLight);

// 3. DAT.GUI 展現參數控制 (展示重點: 重力感、風速感與層級開關)
const gui = new dat.GUI({ autoPlace: true });

```

```
const params = {
  gravityTension: 1.0,
  windSpeed: 0.5,
  layerOffset: 0.02,
  sheenColor: '#88c0d0',
  toggleComparison: false
};

gui.add(params, 'gravityTension', 0.0, 2.5).name('受力重力感 (Gravity)').onChange(v => {
  dressMat.uniforms.uGravityTension.value = v;
});
gui.add(params, 'windSpeed', 0.0, 2.0).name('飄逸風速 (Wind Speed)').onChange(v => {
  dressMat.uniforms.uWindSpeed.value = v;
});
gui.add(params, 'layerOffset', 0.005, 0.08).name('套娃層級間距 (SDF Offset)').onChange(v => {
  dress.scale.set(1 + v, 1, 1 + v);
});
gui.addColor(params, 'sheenColor').name('面料絹光 (Sheen)').onChange(c => {
  dressMat.uniforms.uSheenColor.value.set(c);
});

// 4. Render Loop
const clock = new THREE.Clock();
function animate() {
  requestAnimationFrame(animate);
  const elapsedTime = clock.getElapsedTime();

  dressMat.uniforms.uTime.value = elapsedTime;
  controls.update();
  renderer.render(scene, camera);
}
animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

六、測試與展示驗證流程 (Verification Roadmap)

階段 / 步驟	開發任務	震撼展示點 (Dazzling Presentation)
Phase 1: 基礎環境與微褶皺	佈署 Codespace、導入核心 Custom Shader 與受力權重計算。	拖動「受力重力感」拉桿，肩頸處即時產生微褶皺與法線位移，展現真實重量感。
Phase 2: 套娃層級 (SDF) 防穿模	加入 Layered SDF 距離場動態 Offset，支援多層開啟。	動態切換三層衣服（內衣/襯衫/大衣），外層布料順暢撐起，零穿模現象。
Phase 3: 網頁動態 UI 與效能面板	加入 60 FPS 監控儀表板、對比分割畫面與進階 PBR 面料切換。	展示同等質量下，Web 端運算速度超越 CLO3D 傳統離線物理算圖 10 倍以上。

 Codespace Agent 啟動命令指南

將此 PDF 文件放入專案根目錄後，直接對 GitHub Codespace Agent 輸入：

```
"Please read architecture_plan.pdf and implement the entire cloth simulation PoC project strictly following the step-by-step code and project structure outlined inside."
```